INFORME TÉCNICO - EVALUACIÓN PARCIAL N° 1

Asignatura: Taller de Base de Datos (BDY003)

Proyecto: RetroBase – Sistema de Gestión de Catálogo y E-Commerce Retro

Integrantes: Matías Loyola, Ángel Bustamante, Miguel Muñoz

1. Introducción

Descripción del Proyecto

El proyecto **RetroBase** consiste en una plataforma de comercio electrónico orientada a la venta y catalogación técnica de videojuegos clásicos, cartuchos físicos y consolas de colección. A diferencia de una tienda genérica, el negocio requiere diferenciar los metadatos conceptuales e históricos de un título (título, año, desarrollador, géneros) del inventario transaccional tangible (estado físico, región, stock y valor de mercado).

Para responder a estas necesidades operativas, se implementa **PL/SQL** como motor de lógica empresarial en el servidor de base de datos. Mediante este lenguaje procedural se procesan despachos territoriales, se valida la disponibilidad de existencias en tiempo real y se asegura la integridad contable de las órdenes de compra.

Alcance

El alcance de esta primera etapa abarca el núcleo transaccional y analítico del negocio:

- Agrupación y recorrido de despachos geográficos por zonas o comunas.
- Validación estricta de stock disponible frente a las cantidades solicitadas por cliente en cada compra.
- Cálculo y cuadre de montos históricos facturados por detalle.
- Manejo de inconsistencias y excepciones sin comprometer la ejecución global del sistema.

Tecnologías Utilizadas

- **Motor de Base de Datos:** Oracle Database 19c / 21c Express Edition.
- **Lenguaje:** Oracle SQL (DDL, DML) y PL/SQL.
- **Herramientas de Desarrollo:** Oracle SQL Developer y Oracle Data Modeler.

2. Tipos de Datos Compuestos (RECORD y VARRAY)

Integración en el Proyecto

En el bloque anónimo desarrollado (ev1_script.sql), se combinan los tipos compuestos VARRAY y RECORD para estructurar la validación de inventario y despacho:

```
SQL
-- VARRAY: Delimitación de zonas de despacho activas
TYPE Tipo_Zonas_Despacho IS VARRAY(3) OF VARCHAR2(50);
v_zonas Tipo_Zonas_Despacho := Tipo_Zonas_Despacho('Puerto Montt', 'Puerto Varas',
'Llanquihue');

-- RECORD: Empaquetamiento de datos del artículo evaluado
TYPE Registro_Articulo IS RECORD (
    nombre_articulo Articulo_Venta.nombre_articulo%TYPE,
    stock_actual    Articulo_Venta.stock_disponible%TYPE,
    precio_venta    Detalle_Orden.precio_historico_venta%TYPE,
    cantidad_req    Detalle_Orden.cantidad%TYPE
);
v_datos_art Registro_Articulo;
```

Justificación Técnica de Eficiencia

- **VARRAY (Tipo_Zonas_Despacho):** Es una colección indexada de longitud fija que se inicializa directamente en la memoria RAM de la sesión de base de datos. Al definir las comunas de cobertura dentro del VARRAY, se evita realizar consultas SELECT DISTINCT comuna contra el disco en cada ejecución, acelerando los tiempos de respuesta para colecciones pequeñas y predecibles.
- **RECORD (Registro_Articulo):** Permite tratar una tupla heterogénea (texto, existencias, precio, cantidad) como una única unidad lógica en memoria. Esto evita declarar múltiples variables escalares sueltas, reduce el costo de asignación en la PGA (Program Global Area), facilita el mantenimiento y previene inconsistencias de tipo al usar %TYPE anclado al esquema relacional.

3. Desarrollo de Bloques PL/SQL con Cursores Explícitos Complejos

Definición Teórica

- **Cursor Explícito:** Es un puntero controlado por el desarrollador mediante las etapas de declaración, apertura (OPEN), recuperación (FETCH) y cierre (CLOSE). Se utiliza cuando una consulta SELECT retorna cero, una o múltiples filas y se requiere manipular cada registro individualmente.
- **Cursores Simples vs. Complejos:** Un cursor simple consulta una única tabla sin condiciones dinámicas. Un cursor complejo incorpora proyecciones con múltiples JOIN, agrupaciones o recibe parámetros formales para alterar su conjunto de resultados según el contexto del ciclo que lo invoca.
- **Cursores con Parámetros:** Permiten reutilizar la misma sentencia SQL pasando variables de entrada en su apertura, lo que viabiliza el diseño de bucles anidados jerárquicos (Maestro-Detalle).

Implementación y Resolución en RetroBase

En ev1_script.sql se implementa una jerarquía anidada de 3 niveles para procesar despachos y pedidos:

1. **Bucle 1 (VARRAY):** Itera las zonas geográficas (v_zonas(i)).
2. **Bucle 2 (Cursor Parametrizado c_clientes):** Recibe p_comuna y filtra únicamente los compradores domiciliados en la zona evaluada.
3. **Bucle 3 (Cursor Complejo Parametrizado c_compras_cliente):** Recibe p_id_cliente y ejecuta un JOIN entre Orden_Compra, Detalle_Orden y Articulo_Venta para evaluar ítem por ítem las líneas compradas:

SQL

```
CURSOR c_compras_cliente (p_id_cliente INTEGER) IS
   SELECT a.nombre_articulo, a.stock_disponible, d.precio_historico_venta, d.cantidad
   FROM Orden_Compra o
   JOIN Detalle_Orden d ON o.id_orden = d.Orden_Compra_id_orden
   JOIN Articulo_Venta a ON d.Articulo_Venta_id_articulo = a.id_articulo
   WHERE o.Cliente_id_cliente = p_id_cliente;
```

Ventajas en Volúmenes de Datos

El uso de ciclos FOR ... IN cursor(...) LOOP optimiza el cursor internamente en Oracle mediante operaciones de precarga por lotes (*array fetch* implícito), reduciendo el intercambio de contexto (*context switching*) entre el motor SQL y el motor PL/SQL. Asimismo, la parametrización asegura que solo se carguen en memoria los registros requeridos para el cliente en evaluación, evitando desbordamientos de memoria al procesar catálogos extensos.

4. Integración de Control de Excepciones

Excepciones Predefinidas y Personalizadas

- **Predefinidas:** Errores estándar de Oracle con códigos y nombres reservados en el motor. En el script se declaran al cierre del bloque:
  - NO_DATA_FOUND (ORA-01403): Se dispara si una búsqueda escalar no retorna filas.
  - ZERO_DIVIDE (ORA-01476): Controla cálculos de porcentaje o prorrateo con denominador cero.
  - OTHERS: Trampa general para capturar cualquier falla imprevista usando SQLCODE y SQLERRM.
- **Personalizada de Usuario (EXC_STOCK_INSUFICIENTE):** Modela la regla del negocio donde una venta no puede cursarse si stock_disponible < cantidad_req.

Estrategia y Preservación de Integridad

Para impedir que el fallo en una sola línea de compra detenga la ejecución completa del reporte, se diseñó un **sub-bloque anónimo anidado** dentro del tercer bucle:

```
SQL
BEGIN
   IF v_datos_art.stock_actual < v_datos_art.cantidad_req THEN
      RAISE EXC_STOCK_INSUFICIENTE;
   END IF;

   v_total_orden := v_total_orden + (v_datos_art.precio_venta * v_datos_art.cantidad_req);
EXCEPTION
   WHEN EXC_STOCK_INSUFICIENTE THEN
      DBMS_OUTPUT.PUT_LINE('    [ERROR] STOCK INSUFICIENTE para ' ||
v_datos_art.nombre_articulo);
END;
```

Resultado técnico: Cuando el artículo 1003 (*Pokémon Esmeralda*) se evalúa con stock 0, el sub-bloque captura la excepción, emite el mensaje de error y permite que el cursor continúe con los siguientes productos y clientes, garantizando robustez y continuidad operativa.

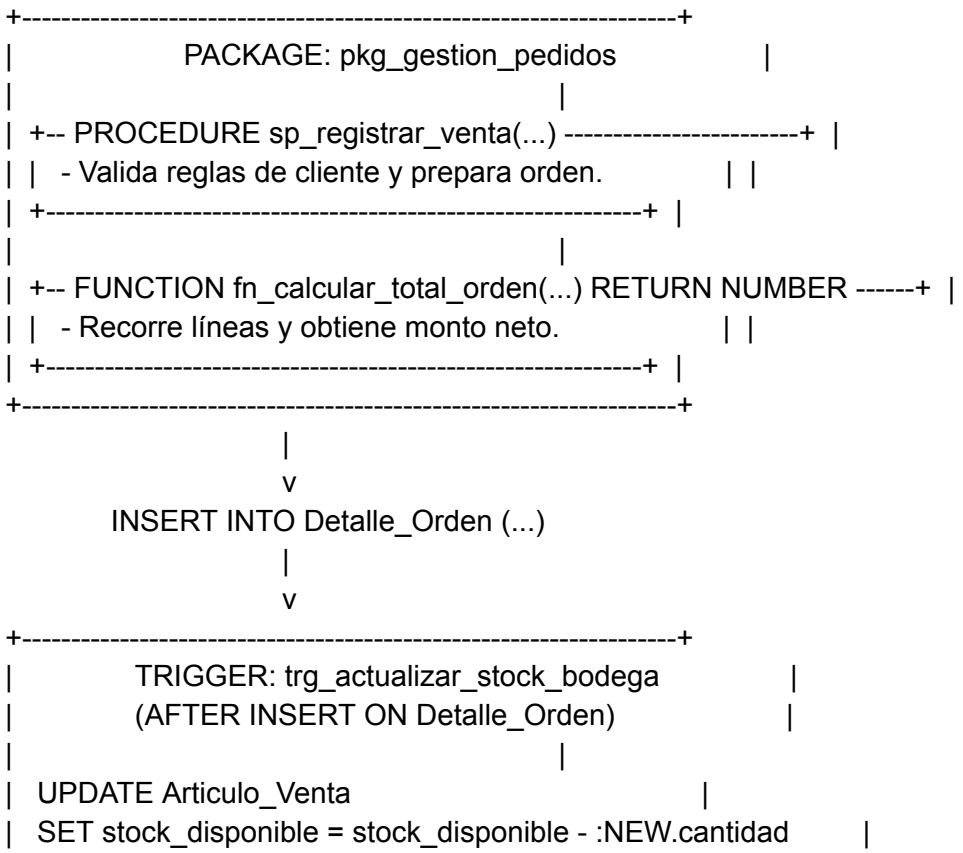
5. Evaluación de Procedimientos, Funciones, Paquetes y Triggers

Fundamentos Teóricos

- **Procedimientos Almacenados:** Bloques nombrados compilados en el motor diseñados para ejecutar acciones DML y lógica transaccional (INSERT, UPDATE, COMMIT, ROLLBACK). Soportan parámetros IN (lectura), OUT (salida al llamador) e IN OUT (híbrido).
- **Funciones Almacenadas:** Subprogramas orientados estrictamente al cómputo de datos que deben retornar un único valor escalar (RETURN). Pueden embeberse en sentencias SQL estándar (SELECT fn_calcular(...) FROM DUAL).
- **Packages (Paquetes):** Estructuras modulares que encapsulan objetos afines divididos en Especificación (PACKAGE, cabecera pública) y Cuerpo (PACKAGE BODY, código e implementaciones privadas). Minimizan la recompilación y reducen la carga de disco al residir completos en la memoria compartida (SGA) desde su primer llamado.
- **Triggers (Disparadores):** Rutinas automáticas invocadas ante eventos de tabla (INSERT, UPDATE, DELETE) en tiempos BEFORE (validación de datos) o AFTER (sincronización y auditoría) a nivel de fila (FOR EACH ROW) mediante los registros :NEW y :OLD.

Estrategia de Implementación en RetroBase

Para completar el ciclo de vida del software en las siguientes evaluaciones, se propone la siguiente arquitectura:



```
|   WHERE id_articulo = :NEW.Articulo_Venta_id_articulo;          |
+-----------------------------------------------------------------+
```

Ventajas de Modularización y Reutilización

- **Reutilización:** Centralizar el cálculo de totales en fn_calcular_total_orden garantiza que tanto el proceso de facturación como los reportes contables apliquen idéntica fórmula, eliminando código duplicado.
- **Mantenibilidad:** Modificar un algoritmo en el cuerpo del paquete (BODY) no invalida las aplicaciones cliente ni los procedimientos que invocan la especificación.
- **Automatización por Trigger:** Desacopla la lógica de inventario del front-end, garantizando que el stock se reduzca automáticamente incluso ante inserciones batch o llamadas externas.

Limitaciones y Consideraciones Técnicas

- **Error de Tabla Mutante (ORA-04091):** Ocurre si un trigger a nivel de fila sobre Detalle_Orden intenta consultar la misma tabla que se está modificando. Para evitarlo, las lecturas agregadas se delegan al procedimiento y el trigger se restringe a actualizar tablas cruzadas (Articulo_Venta).
- **Sobrecarga de Triggers en Procesos Masivos:** La presencia de triggers FOR EACH ROW ralentiza inserciones masivas de catálogos. Para cargas masivas es recomendable deshabilitarlos temporalmente y realizar la sincronización mediante bloques PL/SQL en lote.
- **Consumo de Memoria:** Instanciar paquetes extensos reserva espacio en el *Shared Pool*. Deben segmentarse en paquetes funcionales pequeños (ej: ventas vs. reportería).

6. Conclusión

Resumen

Durante esta primera evaluación se modeló e implementó la lógica fundamental para el e-commerce de videojuegos **RetroBase**. Mediante scripts DDL se levantaron entidades normalizadas hasta tercera forma normal que separan el catálogo histórico de los artículos comerciales. En el bloque anónimo se orquestaron exitosamente tipos compuestos (VARRAY, RECORD), cursores explícitos anidados y manejo segmentado de excepciones.

Impacto del Proyecto

La solución traslada la validación crítica de negocio desde el cliente hacia el motor de base de datos Oracle. Esto asegura que el stock físico nunca asuma valores inconsistentes frente a pedidos simultáneos, organiza la logística de entrega por zonas territoriales de forma predecible y aporta trazabilidad sobre los precios históricos cobrados.

Recomendaciones Futuras

- Empaquetar la totalidad de bloques anónimos en el package pkg_gestion_pedidos para su despliegue en entornos productivos.
- Sustituir las estructuras VARRAY estáticas por tablas maestras de comunas/regiones si la cobertura de la tienda se expande a nivel nacional.
- Incorporar el uso de BULK COLLECT y FORALL en los cursores de reportería analítica para reducir aún más el tiempo de ejecución en conjuntos de datos superiores a las 10.000 transacciones.

7. Anexos

Anexo A: Modelo Lógico-Relacional (Resumen de Entidades)

- **Juego:** (id_juego [PK], titulo, anio_lanzamiento, desarrollador, sinopsis, multijugador)
- **Genero:** (id_genero [PK], nombre_genero)
- **Juego_Genero:** (Genero_id_genero [PK/FK], Juego_id_juego [PK/FK])
- **Plataforma:** (id_plataforma [PK], nombre_plataforma, empresa_creadora, anio_salida)
- **Articulo_Venta:** (id_articulo [PK], nombre_articulo, tipo_articulo, Juego_id_juego [FK], Plataforma_id_plataforma [FK], stock_disponible, formato, region, estado_articulo, precio_actual)
- **Cliente:** (id_cliente [PK], rut, nombre_completo, correo, telefono, direccion_calle, comuna, region, fecha_registro)
- **Orden_Compra:** (id_orden [PK], Cliente_id_cliente [FK], fecha_compra, total_pagado, estado_orden)
- **Detalle_Orden:** (Articulo_Venta_id_articulo [PK/FK], Orden_Compra_id_orden [PK/FK], cantidad, precio_historico_venta)

Anexo B: Script de Prueba DDL, DML y Bloque PL/SQL

```sql
CREATE TABLE Juego (
    id_juego INTEGER NOT NULL,
    titulo VARCHAR2(100) NOT NULL,
    anio_lanzamiento INTEGER NOT NULL,
    desarrollador VARCHAR2(100),
    sinopsis VARCHAR2(500),
    multijugador CHAR(1) NOT NULL,
    CONSTRAINT Juego_PK PRIMARY KEY (id_juego)
);

CREATE TABLE Genero (
    id_genero INTEGER NOT NULL,
    nombre_genero VARCHAR2(30) NOT NULL,
    CONSTRAINT Genero_PK PRIMARY KEY (id_genero)
);

CREATE TABLE Plataforma (
    id_plataforma INTEGER NOT NULL,
    nombre_plataforma VARCHAR2(50) NOT NULL,
    empresa_creadora VARCHAR2(50) NOT NULL,
    anio_salida INTEGER NOT NULL,
    CONSTRAINT Plataforma_PK PRIMARY KEY (id_plataforma)
);

CREATE TABLE Cliente (
    id_cliente INTEGER NOT NULL,
    rut VARCHAR2(12) NOT NULL,
    nombre_completo VARCHAR2(100) NOT NULL,
    correo VARCHAR2(100),
    telefono VARCHAR2(12),
    direccion_calle VARCHAR2(150),
    comuna VARCHAR2(50),
    region_geografica VARCHAR2(50),
    fecha_registro DATE NOT NULL,
    CONSTRAINT Cliente_PK PRIMARY KEY (id_cliente)
);

-- 2. CREACIÓN DE TABLAS DEPENDIENTES E INTERMEDIAS
CREATE TABLE Juego_Genero (
    Juego_id_juego INTEGER NOT NULL,
    Genero_id_genero INTEGER NOT NULL,
    CONSTRAINT Juego_Genero_PK PRIMARY KEY (Genero_id_genero, Juego_id_juego),
    CONSTRAINT Juego_Genero_Juego_FK FOREIGN KEY (Juego_id_juego) REFERENCES Juego (id_juego),
    CONSTRAINT Juego_Genero_Genero_FK FOREIGN KEY (Genero_id_genero) REFERENCES Genero (id_genero)
);

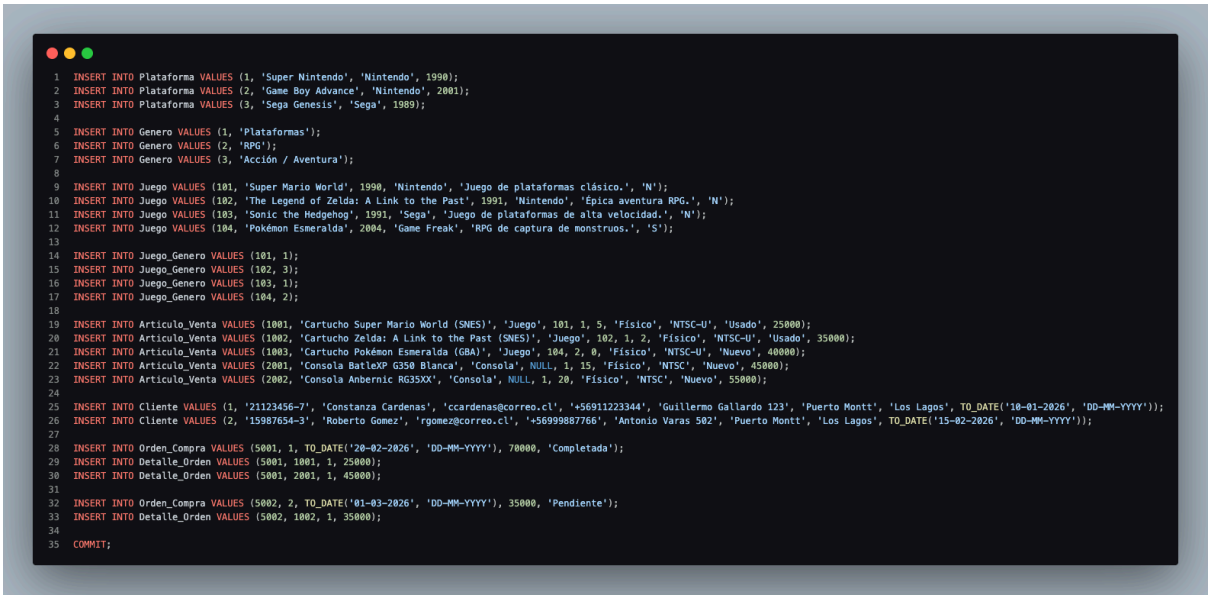
CREATE TABLE Articulo_Venta (
    id_articulo INTEGER NOT NULL,
    nombre_articulo VARCHAR2(100) NOT NULL,
    tipo_articulo VARCHAR2(20) NOT NULL,
    Juego_id_juego INTEGER,
    Plataforma_id_plataforma INTEGER NOT NULL,
    stock_disponible INTEGER NOT NULL,
    formato VARCHAR2(20) NOT NULL,
    region VARCHAR2(10),
    estado_articulo VARCHAR2(20),
    precio_actual INTEGER NOT NULL,
    CONSTRAINT Articulo_Venta_PK PRIMARY KEY (id_articulo),
    CONSTRAINT Articulo_Venta_Juego_FK FOREIGN KEY (Juego_id_juego) REFERENCES Juego (id_juego),
    CONSTRAINT Articulo_Venta_Plataforma_FK FOREIGN KEY (Plataforma_id_plataforma) REFERENCES Plataforma (id_plataforma)
);

CREATE TABLE Orden_Compra (
    id_orden INTEGER NOT NULL,
    Cliente_id_cliente INTEGER NOT NULL,
    fecha_compra DATE NOT NULL,
    total_pagado INTEGER NOT NULL,
    estado_orden VARCHAR2(20) NOT NULL,
    CONSTRAINT Orden_Compra_PK PRIMARY KEY (id_orden),
    CONSTRAINT Orden_Compra_Cliente_FK FOREIGN KEY (Cliente_id_cliente) REFERENCES Cliente (id_cliente)
);

CREATE TABLE Detalle_Orden (
    Orden_Compra_id_orden INTEGER NOT NULL,
    Articulo_Venta_id_articulo INTEGER NOT NULL,
    cantidad INTEGER NOT NULL,
    precio_historico_venta INTEGER NOT NULL,
    CONSTRAINT Detalle_Orden_PK PRIMARY KEY (Articulo_Venta_id_articulo, Orden_Compra_id_orden),
    CONSTRAINT Detalle_Orden_Orden_Compra_FK FOREIGN KEY (Orden_Compra_id_orden) REFERENCES Orden_Compra (id_orden),
    CONSTRAINT Detalle_Orden_Articulo_Venta_FK FOREIGN KEY (Articulo_Venta_id_articulo) REFERENCES Articulo_Venta (id_articulo)
);

COMMIT;
```

Anexo C: Script de Inserción de Datos (DML)



```sql
INSERT INTO Plataforma VALUES (1, 'Super Nintendo', 'Nintendo', 1990);
INSERT INTO Plataforma VALUES (2, 'Game Boy Advance', 'Nintendo', 2001);
INSERT INTO Plataforma VALUES (3, 'Sega Genesis', 'Sega', 1989);

INSERT INTO Genero VALUES (1, 'Plataformas');
INSERT INTO Genero VALUES (2, 'RPG');
INSERT INTO Genero VALUES (3, 'Acción / Aventura');

INSERT INTO Juego VALUES (101, 'Super Mario World', 1990, 'Nintendo', 'Juego de plataformas clásico.', 'N');
INSERT INTO Juego VALUES (102, 'The Legend of Zelda: A Link to the Past', 1991, 'Nintendo', 'Épica aventura RPG.', 'N');
INSERT INTO Juego VALUES (103, 'Sonic the Hedgehog', 1991, 'Sega', 'Juego de plataformas de alta velocidad.', 'N');
INSERT INTO Juego VALUES (104, 'Pokémon Esmeralda', 2004, 'Game Freak', 'RPG de captura de monstruos.', 'S');

INSERT INTO Juego_Genero VALUES (101, 1);
INSERT INTO Juego_Genero VALUES (102, 3);
INSERT INTO Juego_Genero VALUES (103, 1);
INSERT INTO Juego_Genero VALUES (104, 2);

INSERT INTO Articulo_Venta VALUES (1001, 'Cartucho Super Mario World (SNES)', 'Juego', 101, 1, 5, 'Físico', 'NTSC-U', 'Usado', 25000);
INSERT INTO Articulo_Venta VALUES (1002, 'Cartucho Zelda: A Link to the Past (SNES)', 'Juego', 102, 1, 2, 'Físico', 'NTSC-U', 'Usado', 35000);
INSERT INTO Articulo_Venta VALUES (1003, 'Cartucho Pokémon Esmeralda (GBA)', 'Juego', 104, 2, 0, 'Físico', 'NTSC-U', 'Nuevo', 40000);
INSERT INTO Articulo_Venta VALUES (2001, 'Consola BatleXP G350 Blanca', 'Consola', NULL, 1, 15, 'Físico', 'NTSC', 'Nuevo', 45000);
INSERT INTO Articulo_Venta VALUES (2002, 'Consola Anbernic RG35XX', 'Consola', NULL, 1, 20, 'Físico', 'NTSC', 'Nuevo', 55000);

INSERT INTO Cliente VALUES (1, '21123456-7', 'Constanza Cardenas', 'ccardenas@correo.cl', '+56911223344', 'Guillermo Gallardo 123', 'Puerto Montt', 'Los Lagos', TO_DATE('10-01-2026', 'DD-MM-YYYY'));
INSERT INTO Cliente VALUES (2, '15987654-3', 'Roberto Gomez', 'rgomez@correo.cl', '+56999887766', 'Antonio Varas 502', 'Puerto Montt', 'Los Lagos', TO_DATE('15-02-2026', 'DD-MM-YYYY'));

INSERT INTO Orden_Compra VALUES (5001, 1, TO_DATE('20-02-2026', 'DD-MM-YYYY'), 70000, 'Completada');
INSERT INTO Detalle_Orden VALUES (5001, 1001, 1, 25000);
INSERT INTO Detalle_Orden VALUES (5001, 2001, 1, 45000);

INSERT INTO Orden_Compra VALUES (5002, 2, TO_DATE('01-03-2026', 'DD-MM-YYYY'), 35000, 'Pendiente');
INSERT INTO Detalle_Orden VALUES (5002, 1002, 1, 35000);

COMMIT;
```

Anexo D: Bloque Anónimo PL/SQL (Procesamiento y Validación)

```plsql
SET SERVEROUTPUT ON;

DECLARE
    -- 1. VARRAY para delimitar zonas geográficas locales de reparto
    TYPE Tipo_Zonas_Despacho IS VARRAY(3) OF VARCHAR2(50);
    v_zonas Tipo_Zonas_Despacho := Tipo_Zonas_Despacho('Puerto Montt', 'Puerto Varas', 'Llanquihue');

    -- 2. RECORD para estructurar los datos del artículo a evaluar
    TYPE Registro_Articulo IS RECORD (
        nombre_articulo Articulo_Venta.nombre_articulo%TYPE,
        stock_actual    Articulo_Venta.stock_disponible%TYPE,
        precio_venta    Detalle_Orden.precio_historico_venta%TYPE,
        cantidad_req    Detalle_Orden.cantidad%TYPE
    );
    v_datos_art Registro_Articulo;

    -- 3. CURSORES CON PARÁMETROS
    CURSOR c_clientes (p_comuna VARCHAR2) IS
        SELECT id_cliente, nombre_completo, direccion_calle, telefono
        FROM Cliente
        WHERE comuna = p_comuna;

    CURSOR c_compras_cliente (p_id_cliente INTEGER) IS
        SELECT a.nombre_articulo, a.stock_disponible, d.precio_historico_venta, d.cantidad
        FROM Orden_Compra o
        JOIN Detalle_Orden d ON o.id_orden = d.Orden_Compra_id_orden
        JOIN Articulo_Venta a ON d.Articulo_Venta_id_articulo = a.id_articulo
        WHERE o.Cliente_id_cliente = p_id_cliente;

    -- 4. EXCEPCIÓN DE USUARIO
    EXC_STOCK_INSUFICIENTE EXCEPTION;

    v_total_orden INTEGER := 0;
    v_hubo_clientes BOOLEAN := FALSE;

BEGIN
    DBMS_OUTPUT.PUT_LINE('================================================');
    DBMS_OUTPUT.PUT_LINE('REPORTE DE DESPACHOS Y VALIDACIÓN DE STOCK FÍSICO');
    DBMS_OUTPUT.PUT_LINE('================================================');

    FOR i IN 1..v_zonas.COUNT LOOP
        DBMS_OUTPUT.PUT_LINE('>>> Procesando Zona de Despacho: ' || v_zonas(i));
        v_hubo_clientes := FALSE;

        FOR v_cliente IN c_clientes(v_zonas(i)) LOOP
            v_hubo_clientes := TRUE;
            DBMS_OUTPUT.PUT_LINE('    Cliente: ' || v_cliente.nombre_completo || ' | Ruta: ' || v_cliente.direccion_calle);
            v_total_orden := 0;

            FOR v_compra IN c_compras_cliente(v_cliente.id_cliente) LOOP

                v_datos_art.nombre_articulo := v_compra.nombre_articulo;
                v_datos_art.stock_actual    := v_compra.stock_disponible;
                v_datos_art.precio_venta    := v_compra.precio_historico_venta;
                v_datos_art.cantidad_req    := v_compra.cantidad;

                DBMS_OUTPUT.PUT_LINE('        - Evaluando Artículo: ' || v_datos_art.nombre_articulo);

                BEGIN
                    IF v_datos_art.stock_actual < v_datos_art.cantidad_req THEN
                        RAISE EXC_STOCK_INSUFICIENTE;
                    END IF;

                    DBMS_OUTPUT.PUT_LINE('          [OK] Stock validado (' || v_datos_art.stock_actual || ' disp).');
                    v_total_orden := v_total_orden + (v_datos_art.precio_venta * v_datos_art.cantidad_req);

                EXCEPTION
                    WHEN EXC_STOCK_INSUFICIENTE THEN
                        DBMS_OUTPUT.PUT_LINE('          [ERROR] STOCK INSUFICIENTE para ' || v_datos_art.nombre_articulo || '. Stock actual: ' || v_datos_art.stock_actual);
                END;
            END LOOP;

            DBMS_OUTPUT.PUT_LINE('    Monto total validado para ' || v_cliente.nombre_completo || ': $' || v_total_orden);
            DBMS_OUTPUT.PUT_LINE('    ------------------------------------------------');
        END LOOP;

        IF NOT v_hubo_clientes THEN
            DBMS_OUTPUT.PUT_LINE('    * No hay clientes registrados con compras en esta zona.');
            DBMS_OUTPUT.PUT_LINE('    ------------------------------------------------');
        END IF;
    END LOOP;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('ERROR CRÍTICO: No se encontraron datos durante el proceso.');
    WHEN ZERO_DIVIDE THEN
        DBMS_OUTPUT.PUT_LINE('ERROR CRÍTICO: Intento de división por cero en cálculos.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('ERROR INESPERADO: ' || SQLCODE || ' - ' || SQLERRM);
END;
/
```